

Vietnamese GraphRAG

Multi-hop Question Answering over Wikipedia

Capstone Review 2 — FPT University SE+AI

Huynh Quoc Trung (QE180038) Nhu Quang Anh (QE180005) Pham Ngo Dinh Khoi
(QE180110) Truong Dinh Thien (QE180018)

FPT University

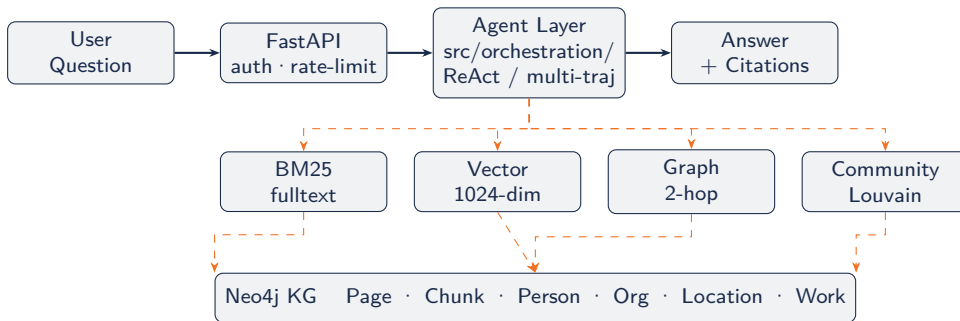
Supervisors: Lê Trung Hiếu Đặng Nguyên Bình

Week 8 — July 2026

Table of Contents

- ❶ System Architecture
- ❷ Deployment & Configuration
- ❸ Ingestion & Query Dataflow
- ❹ Graph Schema (ERD)
- ❺ Physical Design
- ❻ Job State Machine
- ❼ Screen Design
- ❽ Technology Choices
- ❾ Coding Quality & Patterns
- ❿ Algorithm Complexity
- ⓫ Data Pipeline (Corpus Filtering)
- ⓬ Data Pipeline (Schema v0.4)
- ⓭ Data Pipeline (Reasoning Types)
- ⓮ Data Pipeline (QA & QC)
- ⓯ Model Training
- ⓰ Evaluation Results
- ⓱ Team & Timeline

1. System Architecture



Deployment: systemd Neo4j 5.x · uvicorn FastAPI · Python 3.12 · local GPU

2. Deployment & Configuration

Component	Runtime	Start Command
Neo4j 5.x	systemd	<code>sudo systemctl start neo4j</code>
FastAPI	uvicorn	<code>uv run uvicorn src.main:app</code>
Gradio Demo	Python	<code>uv run python src/app_grad</code>
MCP Server	Python	<code>uv run python -m src.mcp_p</code>

All config via pydantic-settings

- NER_BACKEND (simple/underthesea/wikilink/...)
- EMBEDDING_BACKEND (gemini/local)
- MODEL_MODE (api/local)
- WRRF_WEIGHT_BM25/VECTOR/GRAPH
- AGENT_N_TRAJECTORIES
- NEO4J_URI / APP_API_KEY

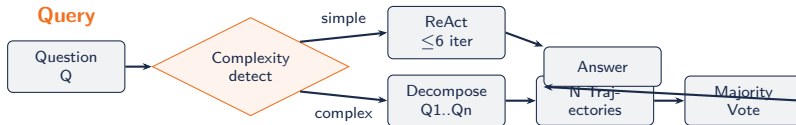
*Zero hard-coded credentials —
.env.example provided*

3. Ingestion & Query Dataflow

Ingestion



Query



4. Graph Schema (ERD)

- (**:Page** {id, title, url, text, source})
- (**:Chunk** {id, text, embedding[1024], seq_num, page_id})
- (**:Person/:Org/:Location/:Work** {id, name, aliases[]})
- (**:Community** {id, level, summary, member_ids[]})

Edges: HAS_CHUNK, MENTIONS_*, LINKS_TO

Relations: FOUNDED_BY, LOCATED_IN, BORN_IN, MEMBER_OF, PART_OF, CREATED_BY

Constraints (UNIQUE)

page.id chunk.id entity.id community.id

Indexes

- Fulltext on chunk.text (BM25)
- Vector on chunk.embedding (1024-dim cosine)
- Btree on entity.name

5. Physical Design

Schema Materialisation

neo4j_client.py creates all constraints and indexes programmatically on startup.
setup_neo4j_schema.py for standalone setup. No manual DDL.

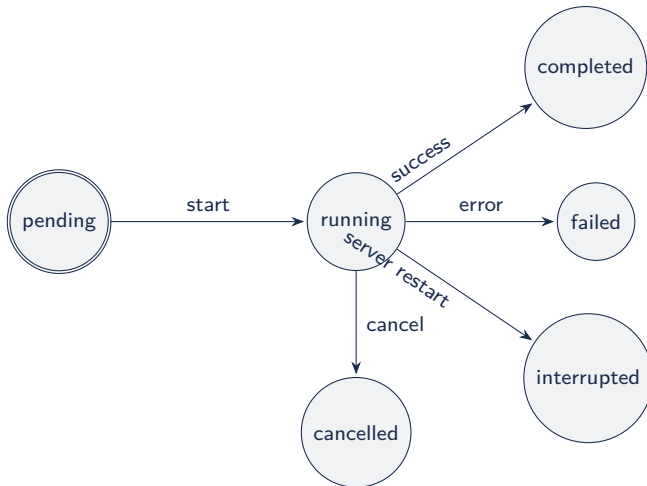
Batch Write Pattern (UNWIND)

```
UNWIND $rows AS row
MERGE (c:Chunk {id: row.id})
SET c += row.props
```

Field Specification

Field	Type	Constraint
id	STRING	UNIQUE
text	STRING	indexed fulltext
embedding	FLOAT[1024]	vector cosine
sequence_number	INTEGER	—
page_id	STRING	references Page.id

6. Job State Machine



7. Screen Design

Interface	Technology	Key Features
Gradio Demo	src/app_gradio.py	Interactive QA, real inference, no mocked responses
GraphPulse Dashboard	src/dashboard/	SVG charts, query logs, KG stats, keyboard nav (/ focus, Esc clear), focus-visible a11y
REST API	FastAPI /query /ingest	OpenAPI docs, API key auth, rate limiting 120 req/min
MCP Server	src/mcp_pkg/	Claude Desktop integration, 6 tools exposed

Real pipeline on every interface

All interfaces run the real NER→embed→WRRF→rerank→LLM pipeline — no fake or hard-coded demo paths.

8. Technology Choices

Layer	Technology	Rationale
Backend	FastAPI + Python 3.12	Async, Pydantic v2, OpenAPI auto-docs
Graph DB	Neo4j 5.x	Vector index + fulltext + Cypher in one DB
Embeddings	GreenNode-Embedding-Large-VN	Best MAP@5 for Vietnamese (1024-dim)
NER	ViDeBERTa/wikilink — 6 backends	Pluggable; wikilink best for bulk (F1=46.9%)
Reranker	BAAI/bge-reranker-v2-m3	Cross-encoder, multilingual, no fine-tuning needed
LLM	Vi-Qwen2-7B-RAG 4-bit NF4	Local-first, Vietnamese-specific, quantized
Toolchain	uv + ruff + mypy + pre-commit	Modern Python: fast deps, lint, type-check

9. Coding Quality & Design Patterns

Coding Conventions ✓

- ruff + ruff-format enforced via pre-commit hooks
- mypy strict type-checking
- CI runs lint + typecheck + test
- snake_case / SCREAMING_SNAKE_CASE / _idx/_ft naming enforced

Config & Secrets ✓

All secrets via `pydantic_settings.BaseSettings` from `.env`. `.env.example` provided. No hard-coded credentials anywhere in source.

Pattern	Applied In	Benefit
Strategy	NER backends via <code>NER_BACKEND</code> env	Swap NER without code change
Singleton	Neo4j driver · Local LLM lazy-load	Single connection pool

10. Algorithm Complexity

Problem Definition

$Q \rightarrow$ retrieve $\{P_1 \dots P_n\}$ from KG $G \rightarrow$ generate A + citations. Complexity levels: 1-hop (lookup), 2-hop (bridge), 3-hop (fan-out).

Algorithm	Purpose	Citation
WRRF 4-signal fusion	BM25+Vector+Graph+Community	Cormack et al. RRF
Leiden/Louvain	Community detection	Edge et al. 2024
ReAct (≤ 6 iter)	Step-by-step graph reasoning	Yao et al. 2023
Multi-trajectory voting	Robust multi-hop (N parallel)	Thompson et al. 2025
QLoRA ($r=32$)	Text2Cypher fine-tuning	Dettmers et al. 2023

WRRF Formula

$$\text{score}(d) = \sum_i \frac{w_i}{k + \text{rank}_i(d)} \quad k = 60$$

$w_{\text{BM25}} = 0.4, w_{\text{Vector}} = 0.4, w_{\text{Graph}} = 0.2, w_{\text{Community}} = 0.15.$ +20.8% MRR vs best non-graph baseline ✓

11. Data Pipeline — Corpus Filtering

11-Stage Filter: viwiki-20260501

Stage	Count	Drop
Initial dump	1,614,015	—
Redirect filter	1,300,130	−313K
Disambiguation	1,284,671	−15K
List pages	1,279,292	−5K
Year/date pages	1,276,304	−3K
Dup-line filter	1,275,171	−1K
Word count pre	414,397	−861K
Orphan filter	397,438	−17K
Word count final	357,747	−40K
Stop words	357,736	−11
Exact dedup	357,714	−22
MinHash dedup	353,498	−4K

Key Design Choices

- Word count pre-filter: largest single drop (−861K stubs)
- Orphan filter: removes unlinked articles (−17K)
- MinHash dedup: Jaccard ≥ 0.85 threshold
- **No** punctuation-ratio filter: Wikipedia tables/lists naturally break the 60% threshold; dropped after analysis

Result

1,614,015 → **353,498** articles
(21.9% of original dump retained)

Snapshot: viwiki-20260501

12. Data Pipeline — ViWikiHop Schema v0.4

Entry Schema (schema v0.4)

Field	Description
_id	hw- / syn-mh- / uvq-
question	Vietnamese question
answer	Empty if unanswerable
supporting_facts	Evidence refs
context	Paragraph segments
type	8 reasoning types
level	easy / medium / hard
source	3 sources
num_hops	≥ 1
is_vi_exclusive	VI-exclusive track
is_impossible	Unanswerable flag
decomposition	Sub-Q/A per hop
plausible_answer	For unanswerable only

3 Data Sources

Source	Description
hw-	Human annotated
syn-mh-	KG walk synthesis
uvq-	ViQuAD 2.0 import

Schema v0.4 Key Invariants

- Single-hop: `decomposition = []`
- Unanswerable: `answer=""`, non-empty `plausible_answer`
- Multi-hop: `|decomposition| = num_hops`
- UIT-ViQuAD: `is_vi_exclusive` always false

13. Data Pipeline — Reasoning Types & Difficulty

8 Reasoning Types

Type	Description
lookup	One article, one fact
bridge	A → bridge entity → B
comparison	Compare ≥ 2 article values
intersection	Entity from ≥ 2 constraints
temporal	Date order / chronology
numerical	Arithmetic or counting
fan-out	Aggregate related entities

3-D Difficulty Rubric

Score = hop_count + lexical_overlap + reasoning_op

Score	Hop	Lexical	ReasonOp
0	1-hop	$\geq 60\%$	lookup
1	2-hop	30–60%	bridge/comparison
2	≥ 3 -hop	$< 30\%$	temporal/numerical/...

Total (0–6)	Level
0–1	easy
2–3	medium
4–6	hard

14. Data Pipeline — QA Generation & QC

QA Generation Pipeline

- ➊ **Bridge Extraction** — follow `[[wikilinks]]` to build article-pair bridges; no Neo4j walk
- ➋ **LLM Sampling** — prompt LLM to generate questions directly from bridge pairs (no templates)
- ➌ **Labelling** — **Gemini 2.5 Flash Lite** reads difficulty rubric and assigns type + level
- ➍ **Verification** — **DeepSeek V3 Pro** independently verifies label and answer grounding

Sample Entry ID

```
syn-mh-viex-b1-a-00001
type=bridge, level=medium, num_hops=2
is_vi_exclusive=true, source=synthetic-multihop
```

Two-Model QC Gate

Model	Role
Gemini 2.5 Flash Lite	Label (type, level, decomposition)
DeepSeek V3 Pro	Verify label + grounding

Gate logic: entry passes only when both models agree on answer grounding; label is taken from Gemini, rejected if DeepSeek flags a mismatch.

Schema QC (post-gate)

- Well-formedness: non-empty fields, schema v0.4
- Grounding: answer in `supporting_facts` verbatim
- Natural phrasing: no “*Theo bài viết*” phrases

18. Team Contribution & Timeline

Member	ID	Modules
Huynh Quoc Trung	QE180038	KG construction · NER (6 backends) · entity resolution · dataset gen
Nhu Quang Anh	QE180005	WRRF retrieval · reranker · evaluation pipeline · SRS docs
Pham Ngo Dinh Khoi	QE180110	ReAct agent · local LLM · MHQA dataset · QLoRA fine-tuning
Truong Dinh Thien	QE180018	FastAPI · job management · Gradio demo · dashboard

Milestone	R1 Status	R2 Status
Core pipeline + SRS	Done ✓	Stable ✓
Full ingestion + ablation	In progress	Done ✓
QLoRA Text2Cypher	Pending	In progress
WRRF tuning + final eval	Pending	In progress
Final report + defense	Pending	Pending

UCP = 96.5 (UUCP=134 · TCF=0.9 · ECF=0.8) → On schedule ✓

Thank You

Q&A

Vietnamese GraphRAG — Multi-hop QA

Appendix: NER Backend Comparison

Backend	Typed F1	Speed	Notes
simple	~20%	Fast	Regex + keyword, no ML
underthesea	~35%	Medium	BIO tagging, general Vietnamese
phonlp	~38%	Slow	PhoNLP + VnCoreNLP segmentation
phobert	~42%	Slow	PhoBERT transformer pipeline
videberta	44%	Medium	ViDeBERTa/NlpHUST electra-base
wikilink	46.9%	Fast	Wikipedia hyperlinks, best for bulk

Appendix: WRRF Detail & Ablation

$$\text{score}(d) = \sum_{i \in S} \frac{w_i}{k + \text{rank}_i(d)} \quad k = 60, \quad \sum w_i \approx 1.15$$

Variant	Signals	MRR	Hit@5	Δ MRR
BM25 only	1	0.35	55%	baseline
+ Vector	2	0.40	60%	+14.3%
+ Graph	3	0.43	63%	+22.9%
+ Community	4	0.4497	64.67%	+28.5%

Appendix: MHQA Generation Pipeline

- ❶ **KG Walk Extraction** — 2-hop, 3-hop, and broken-link walks from Neo4j
(scripts/mhqa_extract_walks.py)
- ❷ **Template QA Generation** — Vietnamese question templates instantiated from walk triples
(scripts/mhqa_generate.py)
- ❸ **LLM Rewrite** — Optional naturalness rewrite via Gemini (scripts/mhqa_generate.py
--rewrite)
- ❹ **5-Stage QC Pipeline** (scripts/mhqa_qc.py):
 - ❶ Well-formedness (length, punctuation, Vietnamese chars)
 - ❷ Grounding (entity_grounded_in_text verification)
 - ❸ Deduplication (MinHash + exact match)
 - ❹ Answerability (answer appears in supporting chunk)
 - ❺ Schema validation (required fields present)
- ❺ **Retrievability Check** (scripts/mhqa_retrievability.py) — confirms each QA pair is retrievable by WRRF pipeline before inclusion